

UniWeb — P2M Platform Readiness

Person-to-Merchant | Done · Partial · Missing | Work Order | Manual UTR | **Missing Technical Steps**

Uniweb Technologist PVT LTD | Brand: UniWeb | v2.0 | 06 August 2026

Document Purpose (Platform-First)

- Focus **ONLY** on UniWeb platform readiness for P2M. Partner keys come later.
- Platform complete → partner onboarding becomes config, not rebuild.
- v2 adds: Missing Technical Steps — exact DB/API/UI work for every gap.

Legend: **DONE** = Built | **PARTIAL** = Incomplete | **MISSING** = Not built

1. What is P2M (Person-to-Merchant)?

Customer pays merchant via UPI QR / Payment Link / Collect / Virtual Account. Money captured → commission accounted → net Available for merchant settlement.

Path: Onboard → Link/QR → Pay → Webhook → Ledger credit → Commission → Available → Settle to bank.

2. Manual UTR — Full Explanation

UTR = Unique Transaction Reference (IMPS/NEFT/RTGS bank reference).

Manual UTR: System does not auto-push money. Human transfers outside system, then types UTR in UniWeb to mark paid. No API keys needed. Slow, error-prone, does not scale. Temporary ops method only.

Replacement: Verified Beneficiary + Auto Payout API → partner returns UTR via webhook → ledger updates. Zero typing.

3. Keys, APIs & Who Provides What

Capability	What needed	Who	When
UPI Collect / Dynamic QR / Links	PG Key ID + Secret + webhook secret	PA (Cashfree/Razorpay/PayU/Decentro)	Live collections
Auto Payout (IMPS/NEFT/UPI)	Payout API keys + IP whitelist	PA or bank H2H / RazorpayX	Auto settlement live
Verified Beneficiary (Penny Drop)	Account Validation API	PA / Bank / Decentro-style	Before auto payout
Auto Reconciliation	Settlement reports API / SFTP	Partner	After live txn
Virtual Accounts	VA create API + credit webhooks	PA / Bank	Multi-VA enable
Platform internal (ledger, UI, KYC)	No external keys — DB + encryption + app config	You (UniWeb)	NOW — build without partner

Keys vs Platform Work

- Build ledger, settlement UI, beneficiary forms, idempotent webhooks, recon matcher **WITHOUT** live keys (sandbox/mocks).
- Manual UTR needs no keys. Auto Payout needs partner keys — keep Manual as fallback until then.

4. Full P2M Status Matrix — Done / Partial / Missing

4.1 Merchant Onboarding & Compliance

#	Item	Status	Detail
1	Merchant signup / login / forgot password	DONE	Portal flows exist
2	Entity KYC (PAN, Aadhaar, GST, Udyam, bank)	DONE	Upload + admin review
3	Video KYC	PARTIAL	Flow present; ops verification harden
4	Merchant agreement + PDF	DONE	Acceptance tracked
5	Team roles (Admin/Finance/Developer/Viewer)	DONE	Role access built
6	Test ↔ Live toggle	PARTIAL	Enforce at API/txn level
7	Merchant bank account add + store	PARTIAL	Form exists; penny-drop weak
8	Beneficiary verification (Penny Drop)	MISSING	Required before auto payout

4.2 Collection Instruments (Core P2M)

#	Item	Status	Detail
1	Payment Links create / share / expire / track	DONE	Merchant can create
2	Static QR generate + print	DONE	Basic QR available
3	Dynamic QR (amount-locked, order-linked)	PARTIAL	Partner order→QR flow needed
4	UPI Collect request	PARTIAL	Live partner collect API
5	Hosted checkout + status page	DONE	Branded pages present
6	Multiple QR per merchant	PARTIAL	Concurrency architecture
7	Virtual Account per merchant/order	PARTIAL	Multi-VA + credit webhook incomplete
8	High-throughput QR (burst / dialer)	MISSING	Queue, rate-limit, multi-VA assign

4.3 Payment Capture, Webhooks & Ledger

#	Item	Status	Detail
1	Webhook endpoints success/fail	PARTIAL	Harden signature + retry
2	Webhook signature verification	PARTIAL	Mandatory on all partners
3	Idempotency (no double credit)	PARTIAL	UNIQUE + claim-first pattern
4	Webhook retry / dead-letter	MISSING	Queue + retry + alert
5	Ledger Available / In-transit / Hold / Reserve	PARTIAL	Strict state machine incomplete
6	Commission deduct at capture	PARTIAL	Real-time auto cut harden
7	Platform vs Merchant wallet separation	PARTIAL	Clear UI + DB separation
8	Failed reason passthrough (mapped language)	MISSING	Mapping dictionary + UI
9	Real-time Available after success	PARTIAL	Immediate credit display

4.4 Settlement & Payout

#	Item	Status	Detail
1	Merchant settlement request UI	PARTIAL	Auto vs manual path unclear
2	Admin settlement / batch	PARTIAL	Still manual-heavy
3	Manual UTR entry + mark paid	DONE	Current ops method

#	Item	Status	Detail
4	Options: Wallet hold vs Bank transfer	PARTIAL	Auto-populate banks needed
5	Verified beneficiary list	MISSING	Verified flag + penny-drop result
6	Auto Payout API (IMPS/NEFT/UPI)	MISSING	Partner keys later; adapter now
7	Payout status webhook → ledger	MISSING	Success/fail/reversal
8	Failed payout reverse to Available	MISSING	Mandatory
9	Maker-Checker high-value payout	MISSING	Admin approve above threshold
10	Settlement schedule (T+0/T+1/on-demand)	PARTIAL	Engine incomplete
11	Admin commission settlement (2 options)	PARTIAL	Same pattern as merchant

4.5 Reconciliation & Reports

#	Item	Status	Detail
1	Transaction list + filters + export	DONE	Core reports present
2	Partner file import / auto-match	MISSING	Recon engine
3	Mismatch queue (amount/UTR/status)	MISSING	Ops exception screen
4	Daily ops summary (GMV, success rate)	PARTIAL	Dashboard partial
5	Invoice / GST PDF	PARTIAL	GST fields harden
6	Audit log money actions	PARTIAL	Expand for settle/payout

4.6 Security & Ops

#	Item	Status	Detail
1	HTTPS + security headers + error pages	DONE	Hardened
2	2FA on payout/settle actions	PARTIAL	Enforce on sensitive money ops
3	Rate limit payment + webhook endpoints	PARTIAL	Needed for burst QR
4	Secrets in env not code	PARTIAL	Config cleanup
5	Support + platform status page	PARTIAL	Basic present

5. Missing Technical Steps (Detailed)

Exact platform work for every MISSING / weak PARTIAL item. No partner keys required unless marked. Implement in order within each block.

5.1 Idempotent Webhook + Ledger Credit

Status: PARTIAL → must complete before live money

- Create table **webhook_events**: id, partner, event_id (UNIQUE), payload_hash, status (claimed/processed/failed), order_id, amount, created_at, processed_at.
- On webhook receive: verify signature → INSERT event_id (ignore duplicate) → if insert ok, claim row → process credit → mark processed.
- If INSERT fails (duplicate key): return HTTP 200 immediately (already handled). Never credit twice.
- Ledger write in same DB transaction as marking webhook processed (or use outbox pattern).
- Add admin screen: re-process failed webhook by id (safe only if status=failed).
- Log raw payload (encrypted/restricted) for 30–90 days for dispute.
- Unit test: same event_id twice → one credit only.

5.2 Ledger State Machine

Status: PARTIAL

- Define balances per merchant: available, in_transit, hold, reserve (columns or ledger_entries table).
- Every money move = ledger_entry row: type (capture/commission/settle/reverse/hold/release), amount, ref_type, ref_id, balance_after.
- Rules: Capture → +available (or +in_transit then release); Settlement start → available→in_transit; Payout success → in_transit→settled; Payout fail → in_transit→available.
- Never update balance without ledger_entry. Forbid negative available.
- Merchant UI reads only from ledger aggregates (not ad-hoc SUM of payments).
- Admin tool: rebuild balance from entries (reconciliation check).

5.3 Settlement UI — Wallet vs Bank (2 options)

Status: PARTIAL

- Settlement page: show Available balance prominently.
- On Settle click: radio/select — (1) Keep in Wallet (no bank move) (2) Transfer to Bank.
- If Bank: dropdown auto-populated from merchant_beneficiaries WHERE status=verified (or unverified with warning).
- Amount field default = full Available; allow partial.
- Create settlement_request row: merchant_id, amount, mode (wallet|bank), beneficiary_id, status=pending.
- Admin commission settle: same two options on platform fee wallet.
- After submit: freeze amount (available→in_transit) until payout completes or manual UTR marked.

5.4 Beneficiary + Penny Drop (platform side now)

Status: MISSING — API keys later; schema + UI now

- Table **merchant_beneficiaries**: id, merchant_id, account_number (encrypted), ifsc, account_name, upi_id (nullable), status (unverified/pending/verified/failed), verify_ref, verified_at, last_error.
- Merchant UI: Add bank / Add UPI → save as unverified.
- Admin/Merchant button Verify → calls Account Validation adapter (mock in dev; real API when keys).
- Store verify response name match score; auto status=verified if match rules pass.
- Only verified beneficiaries selectable for auto payout; unverified allowed only for Manual UTR path.
- Soft-delete / disable beneficiary; never hard-delete if used in past payouts.

5.5 Payout Job + ManualUTR vs ApiPayout Adapters

Status: MISSING — build adapters without live keys

- Table **payout_jobs**: id, settlement_request_id, merchant_id, amount, beneficiary_id, channel (IMPS/NEFT/UPI), status (queued/processing/success/failed/reversed), utr, partner_ref, failure_code, failure_reason, attempts, created_at, updated_at.
- Interface PayoutAdapter: submit(job) → result; getStatus(partner_ref) → result.
- ManualUTRAdapter: does not call bank; admin enters UTR → mark success (current behaviour).
- ApiPayoutAdapter: HTTP to partner payout API (implement with mock/sandbox first).
- Worker/cron: pick queued jobs → processing → adapter.submit → on success store UTR + settle ledger; on fail reverse to available + reason.
- Idempotency key on payout submit = payout_jobs.id (partner must accept).
- Maker-checker: if amount >= threshold, status=awaiting_approval until admin approves → then queued.

- Webhook/callback from partner for async status → update job + ledger (idempotent).

5.6 Failed Reason Mapping

Status: MISSING

- Table or config **reason_map**: partner, error_code, merchant_message_en, merchant_message_hi, severity.
- On webhook/API fail: map code → message; store both raw_code and mapped_message on transaction/payout_job.
- Merchant UI shows only mapped_message (Hindi/English). Admin sees raw + mapped.
- Default fallback: "Payment could not be completed. Please try again or contact support."
- Seed common codes: INSUFFICIENT_FUNDS, BANK_TIMEOUT, RISK_REJECTED, INVALID_VPA, DEEMED_SUCCESS handling.

5.7 Webhook Retry + Dead Letter

Status: MISSING

- If process throws after claim: mark `webhook_events` `status=failed`, `error_text`, `next_retry_at`.
- Retry worker: exponential backoff (1m, 5m, 30m, 2h); max N attempts → `dead_letter` flag.
- Admin Dead Letter queue: list failed events, payload preview, Re-process / Discard.
- Alert (email/Telegram) when `dead_letter` count > 0.
- Never retry success already processed (idempotent guard).

5.8 Reconciliation Engine (stub → full)

Status: MISSING

- Table **recon_batches**: `id`, `partner`, `file_name`, `period_date`, `status`, `uploaded_by`, `created_at`.
- Table **recon_lines**: `batch_id`, `partner_txn_id`, `utr`, `amount`, `status_in_file`, `matched_order_id`, `match_status` (`matched/amount_mismatch/missing_local/missing_partner`).
- Upload CSV/SFTP parse → insert lines → auto-match on `order_id` / `partner_payment_id` / UTR.
- Ops UI: filter mismatches → resolve (adjust ledger / mark investigated / raise dispute).
- Daily job: if partner API available, pull settlement report instead of manual upload.
- Report: recon success % and open exceptions count on admin dashboard.

5.9 Dynamic QR + Order Binding

Status: PARTIAL

- Internal orders table: `order_id` (UUID), `merchant_id`, `amount`, `status`, `payment_link_id/qr_id`, `partner_order_ref`, `expires_at`.
- Create order → call partner `create-order/create-qr` (sandbox) → store partner ref + QR string/URL.
- Customer pay → webhook carries `partner_order_ref` → map to internal `order_id` → credit once.
- Expire job: mark expired orders; reject late webhooks for expired (or policy: accept + flag).
- Merchant list: filter by order status; show QR/link copy buttons.

5.10 High-Throughput QR (when volume needed)

Status: MISSING

- Pool of Virtual Accounts / Dynamic QR templates per merchant (not one QR for all burst traffic).
- Assign least-loaded VA/QR to new payment intent; release after success/expire.
- Async webhook workers (queue) — HTTP endpoint only enqueues, workers credit ledger.
- Rate-limit per merchant + global; return friendly busy if over cap.
- Idempotency mandatory (5.1). DB indexes on `event_id`, `order_id`, `merchant_id+created_at`.
- Load test target: define TPS goal; measure webhook→available latency p95.

5.11 Security Hardening for Money Paths

Status: PARTIAL

- Force 2FA step-up on: add beneficiary, settle, approve payout, export full ledger.
- CSRF + same-site cookies on all POST money actions.
- Rate-limit settle and payout request endpoints per merchant and per admin.
- Encrypt `account_number` / `upi_id` at rest; mask in UI (****1234).
- Immutable audit_log: `actor`, `action`, `entity`, `before_json`, `after_json`, `ip`, `created_at`.

5.12 Test Mode Isolation

Status: PARTIAL

- Every order/payment/payout row has environment flag: `test` | `live`.
- Test webhook keys only credit test ledger; live keys only live ledger.
- Merchant Test toggle cannot read/write live balances.
- Admin dashboard filter Test vs Live; never sum mixed.

6. Recommended Work Order (Platform-First)

Partner keys only in Phase D. Finish A–C so demo is clean collect → balance → settle.

Phase A — Foundation Lock (Week 1–2) | No live keys

- A1 Ledger state machine + ledger_entries (5.2)
- A2 Idempotent webhook_events + claim-first (5.1)
- A3 Merchant balance UI (Available / In-transit / Hold)
- A4 Settlement UI: Wallet vs Bank + auto-populate beneficiaries (5.3)
- A5 Keep Manual UTR path working (ManualUTRAAdapter)
- A6 Beneficiary table + Unverified/Verified UI (5.4 schema)
- A7 Audit log on all money actions (5.11)
- A8 Reason map table + show mapped fail reason (5.6 seed)

Phase B — Collection Completeness (Week 2–3) | Sandbox keys OK

- B1 Orders + Dynamic QR / Link bound to order_id (5.9)
- B2 Payment status machine Created→Success/Failed/Expired
- B3 Webhook retry + dead letter queue (5.7)
- B4 Commission at capture; platform fee wallet separate
- B5 Merchant reports by day/method + export
- B6 Test/Live isolation enforced (5.12)

Phase C — Settlement Automation Ready (Week 3–4)

- C1 payout_jobs table + status lifecycle (5.5)
- C2 ManualUTRAAdapter + ApiPayoutAdapter interface (mock API)
- C3 Success → UTR store + ledger settle; Fail → reverse + reason
- C4 Maker-checker threshold for large payouts
- C5 Admin commission settle same 2 options
- C6 Recon upload + match stub (5.8)

Phase D — Partner Plug-in (after A–C)

- D1 Live UPI Collect + Dynamic QR (one primary PA)
- D2 Production webhook signature + IP allowlist
- D3 Penny Drop API on beneficiary verify
- D4 Auto Payout IMPS/UPI for verified beneficiaries
- D5 Daily settlement report auto-match
- D6 Manual UTR = emergency override only

7. Current P2M Readiness Score (Platform Only)

Area	/10	Comment
Merchant onboarding + KYC	7.5	Strong base
Payment Link + Basic QR	6.5	Dynamic/high-TPS pending
Webhook + Idempotent ledger	4.5	Harden before live money
Settlement UI + Manual UTR	6.0	Ops works; auto missing
Auto Payout + Verified beneficiary	2.0	Adapters now; keys later
Reconciliation engine	2.5	Reports yes; auto-match no
High-throughput QR	2.5	Planned not built
Overall P2M Platform	4.5	Finish Phases A–C for production P2M

8. Executive Summary

Bottom Line

- P2M foundation exists: portal, KYC, links, basic QR, transactions, Manual UTR settlement.
- Biggest gaps: idempotent webhook→ledger, clear Available balance, payout_jobs + adapters, recon.
- Manual UTR = human transfer + type reference. No keys. Does not scale.
- Auto Payout needs partner keys later — build ApiPayoutAdapter + UI + reverse logic NOW with mocks.
- Section 5 = exact technical steps for every missing piece. Follow Phase A→B→C then D.
- When demo shows collect → balance → settle cleanly, partners give keys faster.

v2.0 · P2M only · Platform-first · Missing Technical Steps included · uniweb.co.in